Human NCLE–NED analyses
Code and reproduction guide

For Tarun Kumar and Quyen Vu | 9 October 2026

This guide supplies the recovered human analysis code, runnable reproduction utilities, the input snapshots, and an independent-review brief. The reported results were recalculated from protein-level rows: all 12 reported result rows matched, and an independent optimizer confirmed the six Model 2 slope rows.

The scope is the human crude and adjusted NCLE–NED association analyses, their cohort/annotation checks, and the disorder-query workflow. No completed mouse/yeast association analysis or refolding simulation is claimed here.

1. Reproduce the reported results first

Extract the ZIP and open its Human_NCLE_reproduction folder in VS Code. Run the following lines in the integrated Terminal, one at a time. These are shell commands; do not paste them into a Python notebook cell. The Terminal working folder must contain scripts/, data/, and requirements.txt.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python scripts/export_complex_annotations.py
python scripts/reproduce_results.py
```

Python 3.12.14 was used for validation. If python3.12 is not installed, choose an installed compatible Python and record its version. The pinned packages are numpy 2.3.5, pandas 2.2.3, scipy 1.17.0, and openpyxl 3.1.5. Installing them needs internet; the two analysis commands above use only local files.

Expected messages: “Recorded-decision reconstruction: PASS” and “Numerical reproduction: PASS”. The result folder is results/frozen_replay/. The runner recomputes estimates from the cohort rows; it reads stored estimates only afterward to compare them. It exits with a failure status if a validation or numerical comparison fails.

Next command	What it checks
python scripts/audit_raw_sources.py	Raw input integrity and filter counts through membrane exclusion; final secreted-source replay is explicitly not run.
python scripts/audit_disorder.py	Processed disorder fractions, query status, interval clipping, and coverage without removing statistics rows.
python scripts/prepare_mobidb_ids.py	The 13,690 pLDDT-qualified broad query IDs and a five-ID test list; no network request.

2. Inputs, coding, and eligibility

Source	Use in this analysis
data/reference_human_analysis.xlsx	Unchanged reviewed final workbook. Primary Cohort and Sensitivity Cohort contain the protein-level rows; Association Results contains the reported comparison targets.
raw_sources/McShane_human_mmc4.xlsx	Human McShane supplement, sheet FinalTable_RPE_. Uses Protein IDs (Uniprot) and Degradation profile; the original kinetics are not refitted here.
Human_AF_combined_20250614.csv	Jiang NCLE records, pipe-separated. Aggregates CCBond records by gene/accession.
Human_Avg_pLDDTs.csv	Jiang’s dated mean pLDDT and SD. Uses the provided averages, rather than a newly downloaded AlphaFold version.

Source	Use in this analysis
AF4_knots.dat	Human knot status; knotted proteins excluded from statistics cohorts.
UP000005640_9606.fasta	Matched accession sequence availability and sequence length. Length is amino-acid residue count.
Historical membrane-keyword FASTA	Exact accession set for membrane exclusion. Filename is preserved under raw_sources/.
Historical secreted-keyword export	Required for a fully independent raw replay of the last filter. It was not recovered for this package; the final cohorts and flags are preserved in the reviewed workbook.
Saved UniProt Evidence and Manual Review	Frozen evidence/decisions reconstruct the final complex covariate. Exported to CSV by export_complex_annotations.py.
data/processed_MobiDB_master.csv	Processed 13,690-protein disorder export. It is not a collection of raw API response JSON files and is not a statistical inclusion criterion.

Join by exact UniProt accession. Do not silently map protein groups, isoforms, historical IDs, or homologs onto one another. The current matching reproduces the historical exact-accession analysis; recovering additional grouped experimental IDs would constitute a revised cohort and must be reported separately.

- Outcome: NED_binary = 1 for NED. In the primary analysis ED = 0 and UN is excluded; in sensitivity, ED and UN = 0. “Sensitivity” means repeating the analysis with UN included in the comparison group.
- Exposure: NCLE = 1 for native non-covalent entanglement; NCLE = 0 for no detected entanglement. In the supplied repository convention, any covalent CCBond record excludes the protein; at least one non-covalent record gives 1; a represented protein with no Boolean entanglement record gives 0. An accession absent from the source remains missing, not 0.
- Use inclusive average pLDDT >=70. Require binary NCLE and a FASTA sequence; remove knots, membrane proteins, and secreted-keyword proteins.
- Complex_member = 1 for an oligomer/protein-complex member; 0 for an explicitly annotated monomer. Unknown or unresolved is missing. Binding alone does not establish an assembly. A protein with documented monomeric and assembled forms is treated as complex-capable under the recorded rule.
- Disorder availability and fraction do not select statistical cohorts. They are relevant to later simulation selection; no disorder cutoff has been agreed here.

Sequential step	Primary	Sensitivity
pLDDT >=70 and selected degradation categories	1043	1836
Usable binary NCLE status	1034	1810
Exclude knotted proteins	1023	1793
Require FASTA sequence	1022	1792
Exclude membrane proteins	725	1230
Exclude secreted proteins: final analysis cohort	705	1191
Model 2: additionally require usable complex annotation	528	901

The supplied McShane human sheet has 3,133 rows: 1,474 ED, 296 NED, and 1,363 UN. ED/NED alone gives 1,770 experimental records before structural matching and QC. The 13,690-protein pLDDT-qualified query master is a broader structural dataset; it is not the statistical cohort.

3. What the calculations do

For the contingency table, a = entangled NED, b = entangled non-NED, c = not-entangled NED, d = not-entangled non-NED. The crude odds ratio is (a × d)/(b × c). Its reported 95% CI is exp[log(OR) ± 1.96 × sqrt(1/a + 1/b + 1/c + 1/d)]. The crude P is a two-sided Fisher exact P. The CI is a log-OR Wald interval, not an exact Fisher confidence interval. All current cells are nonzero.

Cohort	Entangled NED / non-NED	Not entangled NED / non-NED
Primary: NED versus ED	96 / 403	41 / 165
Sensitivity: NED versus ED + UN	96 / 754	41 / 300

Length is standardized within each full analysis cohort as z(length) = (length − cohort mean)/sample SD, using ddof=1. Model 2 then removes rows with missing complex status without restandardizing length. Each logistic model has an intercept and estimates maximum-likelihood coefficients with no penalty.

```
Model 1: logit[P(NED=1)] = intercept + b_NCLE*NCLE + b_L*z(length)
Model 2: logit[P(NED=1)] = intercept + b_NCLE*NCLE
                           + b_C*Complex_member + b_L*z(length)
```

For each fitted term, OR = exp(coefficient). The logistic 95% CI is exp(coefficient ± 1.96 × standard error), using the inverse observed information matrix. P is a two-sided normal Wald P for coefficient = 0. The original implementation uses Newton steps; the additional independent check uses SciPy trust-exact with analytic gradient and Hessian.

Output column	Meaning
Analysis / Method / Term	Which cohort, calculation/model, and predictor the row describes.
N	Number of proteins used in that calculation; missing complex calls reduce Model 2 N.
Coefficient	Fitted log odds ratio for the term. Exponentiating it gives Odds_ratio.
Odds_ratio	NCLE: entangled versus not entangled; complex: member versus explicit monomer; length: a one-SD length increase, holding other model terms fixed.
CI_lower / CI_upper	Lower/upper bounds of the stated 95% Wald interval. An interval crossing 1 includes the no-association OR.
P_value	Crude: Fisher exact. Logistic: Wald. A large P does not prove no effect.

4. Expected results: all reported terms

Primary: NED vs ED

Method	Term	N	OR	95% CI	P
Crude	NCLE	705	0.959	0.637–1.442	0.835
Model 1	NCLE	705	0.953	0.630–1.440	0.818
Model 1	z(length)	705	1.019	0.847–1.225	0.844
Model 2	NCLE	528	0.929	0.599–1.441	0.741
Model 2	Complex member	528	9.156	1.229–68.237	0.031
Model 2	z(length)	528	1.001	0.829–1.208	0.992

Sensitivity: NED vs ED + UN

Method	Term	N	OR	95% CI	P
Crude	NCLE	1191	0.932	0.631–1.375	0.763
Model 1	NCLE	1191	0.950	0.640–1.411	0.800
Model 1	z(length)	1191	0.946	0.776–1.152	0.580
Model 2	NCLE	901	0.902	0.597–1.361	0.623
Model 2	Complex member	901	6.734	0.915–49.584	0.061
Model 2	z(length)	901	0.932	0.760–1.142	0.495

The NCLE estimates are close to 1 and their intervals cross 1 in both cohorts and both adjusted models. The appropriate statement is: “We did not detect a positive NCLE–NED association in the analyzed human cohorts.” These results do not prove absence of an effect or identify the conformational cause of NED.

The primary complex-member estimate is OR 9.16 (95% CI 1.23–68.24, P=0.031); sensitivity is OR 6.73 (0.91–49.58, P=0.061). This is compatible with the complex-membership hypothesis but is imprecise and observational. Primary has 28 explicit monomers, sensitivity 40; just one monomer is NED in each. The estimates do not establish that complex membership causes NED.

5. Complex annotations and diagnostic analyses

The workbook has 1,191 saved UniProt evidence records. Of 205 initially ambiguous records, 28 were provisionally assigned complex/oligomer and 177 remained unresolved. The final primary Model 2 uses 500 complex members + 28 monomers; sensitivity uses 861 + 40. Missing complex status excludes 177 primary and 290 sensitivity proteins from Model 2.

The evidence snapshot records the historical UniProt release and retrieval metadata. It preserves initial labels, evidence text, and final review decisions. The new export utility replays those recorded decisions exactly; it does not recreate an unavailable text classifier. A workflow flag saying a record was processed is not proof of independent human confirmation. The 28 provisional positive calls still merit an independent scientific review.

Diagnostic output	Purpose and expected behavior
restricted_model1.csv	Refit Model 1 on the Model 2 complete-case sample. This separates sample restriction from adding the complex term. NCLE OR: approximately 0.874 primary and 0.859 sensitivity.
provisional_exclusion_results.csv	Drop the provisional positive calls and refit Model 2. N=512/873; NCLE OR approximately 0.912/0.897. Similar current estimates are a sensitivity check, not a guarantee that all unresolved biological calls would leave results unchanged.
model2_fisher_diagnostics.csv	Two-sided Fisher tests: complex versus monomer in known annotations; usable versus missing complex annotation in the full cohort. Availability is associated with NED (OR approximately 6.83/5.89), indicating possible complete-case selection bias.
annotation_status_counts.csv	Counts and NED events by the saved annotation category. Check the sparse explicit-monomer category and unresolved records.
independent_optimizer_comparison.csv	Six Model 2 slope rows independently refitted; 36 metric checks all passed at tolerance 1e-8. The intercept is fitted but is not a reported comparison row.

6. Disorder workflow and the Q8NI36 inconsistency

The archived production query unions available selected curated, homology, missing-residue, prediction, and mobility tracks. It does not choose only a single highest-priority track. Inclusive overlapping/adjacent intervals are merged. Counts are clipped to positions 1 through the length retrieved from UniProt; the output retains the original merged coordinates. Missing annotations or failed requests remain missing in the production output.

The processed broad export contains 13,690 proteins: 12,609 status ok; 1,042 no selected annotation; 31 no selected annotation plus missing length; 8 missing length. The 705-protein primary statistical cohort has 592 available fractions and 113 missing fractions. It remains 705 for crude/Model 1 statistics.

There are 18 interval-coordinate warnings in the broad table. For Q8NI36, length is 895 and intervals are 1–63; 722–734; 807–809; 949–951. Their full union contains 82 residues, but only 79 are within positions 1–895. Thus the stored fraction is 79/895. This explains the arithmetic, while leaving a sequence-coordinate/version mismatch unresolved. Raw API responses and matching sequence versions are needed for a biological resolution; a live query alone may use a newer version.

The audit writes interval_coordinate_warnings.csv and annotated cohort CSVs, but it does not modify source fractions or automatically remove Q8NI36 from statistical analyses. For simulation selection, reconcile the sequence and disorder coordinates before relying on the fraction.

Optional: deliberate live query

A live query is optional and was not rerun here. It requires requests and internet access. Keep refreshed output separate from the frozen evidence. The recoverable archived copy has no confirmed copy of a later laptop IPv4 patch; network behavior may differ from the successful local run.

```
python -m pip install -r requirements-live-query.txt
python scripts/prepare_mobidb_ids.py
python scripts/query_mobidb_disorder_production.py \
  -f results/query_inputs/uniprots_test5.txt \
  -o results/disorder_live_test5.tsv
```

Check the five records and their query_status values before a full 13,690-ID query. Replace the input with results/query_inputs/uniprots_plddt_ge70.txt and choose a new full-output TSV. Repeating the same command resumes successful records and retries the other statuses. The older sample needs different dependencies; it is included for review rather than as the recommended replay command.

7. Fully rebuild the cohorts from raw sources

The current source audit reproduces all filter counts through membrane exclusion (725 primary; 1,230 sensitivity). The recovered knot source gives the same 123 pLDDT-qualified human IDs under both the historical normalization and strict fragment-suffix removal; no additional IDs were found for these sources.

Recover the approved historical secreted-keyword export from the local project. Supply only secreted-positive accessions as a FASTA, JSON list, or one-ID-per-line text file. If starting from a table with status 0/1, restrict to status 1 before exporting IDs. The original loader treats its source as a set of positive IDs, not as a mixed-status table. Never infer the positive set from the difference between final and pre-secreted cohorts.

```
python scripts/audit_raw_sources.py \
  --secreted-source raw_sources/secreted_ids.json \
  --out results/full_source_audit
```

Expected final N: primary 705; sensitivity 1,191. final_membership_comparison.csv should have no added/omitted IDs and all key-field checks should pass. This final secreted-source branch could not be tested against an independent approved export here. The workbook-based numerical replay does pass.

After exporting data/complex_annotations.csv and recovering that secreted source, the complete original analysis command is:

```
python scripts/statistical_analysis.py \
  --mcshane raw_sources/McShane_human_mmc4.xlsx \
  --entanglement raw_sources/Human_AF_combined_20250614.csv \
  --plddt raw_sources/Human_Avg_pLDDTs.csv \
  --knots raw_sources/AF4_knots.dat \
  --proteome-fasta raw_sources/UP000005640_9606.fasta \
  --membrane-fasta raw_sources/uniprotkb_human_keywords_membrane_2025_03_19.fasta \
  --secreted-source raw_sources/secreted_ids.json \
  --complex-annotation data/complex_annotations.csv \
  --output-dir results/full_raw_rebuild
```

The complex annotation CSV is a frozen recorded-decision export, so this command reconstructs the numerical covariate but does not independently reclassify biological evidence. A present-day annotation download should be treated as a separate revised analysis and not used to overwrite this reference.

8. Files to inspect and share

File or folder	What to do with it
README.txt	Start here; short terminal instructions, expected checks, and recovery limits.
scripts/*.py	Individual executable source files. These are the safest copies for execution.
documentation/CODE_COMPENDIUM.txt	Full plain-text collection of current and historical recovered Python code; useful for another LLM. Do not execute this combined file as one script.
documentation/Independent_review_prompt.txt	Current independent-check instructions with the correct cohorts, models, and missing-provenance limits.
results/frozen_replay/association_results.csv	All 12 recalculated result rows, including NCLE, length, and complex terms.
results/frozen_replay/reference_comparison.csv	72 N/estimate/CI/P checks against the final reported workbook rows.
results/raw_source_audit/	Filter comparisons and source hashes; manifest states the final secreted check was not run.
results/disorder_audit/	Fraction arithmetic, 18 coordinate warnings, and cohort coverage without disorder exclusion.
historical_audits/	Recovered older audit scripts and repository notebook. Older 725/606/85-threshold contexts are historical, not current statistical eligibility rules. Simulation preflight is a setup check, not a simulation result.
SOURCE_MANIFEST.json	Source/copy provenance and SHA-256 hashes for supplied files. Run metadata includes software versions and reference hashes.

The local modified ED/NED notebook and the original automatic UniProt text classifier were not recovered. The original statistical analysis script and protein-level snapshots are sufficient for the validated numerical replay. Earlier pilot matching of 20 proteins is not used in these regressions and its exact local selection code is not reconstructed here.

Common execution problems

- “No such file”: verify the Terminal working folder contains scripts/ and data/. You may have opened the separate MobiDB folder instead of this extracted reproduction folder.
- “NameError: master_df”: that name belonged to a running notebook kernel. These standalone scripts load their own input files; they do not depend on notebook execution order.
- zsh parse errors or a function> prompt: Python expressions were pasted into the shell. Press Ctrl+C and run the terminal commands above, or put Python code in a notebook cell.

- Missing packages: activate this folder's .venv and install requirements.txt with python -m pip. Do not install bioservices for the production query script; its requests dependency is separate.
- A numerical FAIL: inspect reference_comparison.csv, the source hashes, and the first traceback. Do not silently edit the benchmark or change unknown complex calls into monomers.

9. Complete copyable current code

Each appendix gives a complete individual file as selectable text, including indentation. Copy only that file's code into a .py file with the shown name. Keep all current scripts together under scripts/ because some import statistical_analysis.py. Individual files in the ZIP avoid copying errors. The code below is identical to those supplied files; visual wrapping in the PDF is not a Python newline.

Appendix 1. statistical_analysis.py

Location: scripts/statistical_analysis.py
Provenance: Recovered original; unchanged

Builds statistical cohorts from raw sources and calculates the crude OR, two-sided Fisher P, and logistic models. With supplied binary complex annotations it also fits Model 2. This is the original calculation implementation imported by the reproduction runner.

```
#!/usr/bin/env python3
"""Reconstruct and analyze the human NCLE/degradation cohorts.

Primary analysis: NED versus ED.
Sensitivity analysis: NED versus ED + UN.

MobiDB disorder is intentionally not used as an inclusion criterion.
"""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import numpy as np
import pandas as pd
import scipy.stats as stats


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument("--mcshane", type=Path, required=True)
    parser.add_argument("--entanglement", type=Path, required=True)
    parser.add_argument("--plddt", type=Path, required=True)
    parser.add_argument("--knots", type=Path, required=True)
    parser.add_argument("--proteome-fasta", type=Path, required=True)
    parser.add_argument("--membrane-fasta", type=Path, required=True)
    parser.add_argument("--secreted-source", type=Path, required=True)
    parser.add_argument("--monomer-fasta", type=Path, action="append", default=[])
    parser.add_argument("--complex-annotation", type=Path)
    parser.add_argument("--output-dir", type=Path, required=True)
    return parser.parse_args()


def normalize_id(value: object) -> str:
    return str(value).strip()


def merge_ccbond(series: pd.Series) -> float:
    values: list[bool] = []
    for item in series.dropna():
        if isinstance(item, (bool, np.bool_)):
            values.append(bool(item))
        elif str(item).strip() in {"True", "1", "1.0"}:
            values.append(True)
```

```
        elif str(item).strip() in {"False", "0", "0.0"}:
            values.append(False)
    if any(values):
        return np.nan
    if values:
        return 1.0
    return 0.0


def parse_fasta(path: Path) -> dict[str, int]:
    lengths: dict[str, int] = {}
    current: str | None = None
    with path.open() as handle:
        for raw_line in handle:
            line = raw_line.strip()
            if not line:
                continue
            if line.startswith(">"):
                parts = line[1:].split("|")
                current = parts[1] if len(parts) >= 3 else parts[0].split()[0]
                current = normalize_id(current)
                lengths.setdefault(current, 0)
            elif current is not None:
                lengths[current] += len(line)
    return lengths


def parse_id_source(path: Path) -> set[str]:
    suffix = path.suffix.lower()
    if suffix in {".fa", ".faa", ".fasta"}:
        return set(parse_fasta(path))
    if suffix == ".json":
        payload = json.loads(path.read_text())
        values = payload.get("secreted", payload) if isinstance(payload, dict) else payload
        return {normalize_id(value) for value in values}
    values: set[str] = set()
    with path.open() as handle:
        for raw_line in handle:
            value = raw_line.strip().split("\t")[0].split(",")[0]
            if value and value.lower() not in {"uniprot", "uniprot_id", "entry"}:
                values.add(normalize_id(value))
    return values


def load_sources(args: argparse.Namespace) -> tuple[pd.DataFrame, dict[str, set[str]], dict[str, int]]:
    mcshane = pd.read_excel(args.mcshane, sheet_name="FinalTable_RPE_")
    mcshane = mcshane[["Protein IDs (Uniprot)", "Degradation profile"]].rename(
        columns={"Protein IDs (Uniprot)": "UniProt_ID", "Degradation profile": "Degradation_profile"}
    )
    mcshane["UniProt_ID"] = mcshane["UniProt_ID"].map(normalize_id)

    plddt = pd.read_csv(args.plddt)[["gene", "<pLDDT>", "std"]].rename(
        columns={"gene": "UniProt_ID", "<pLDDT>": "Average_pLDDT", "std": "pLDDT_SD"}
    )
    plddt["UniProt_ID"] = plddt["UniProt_ID"].map(normalize_id)
    plddt["Average_pLDDT"] = pd.to_numeric(plddt["Average_pLDDT"], errors="coerce")
    plddt["pLDDT_SD"] = pd.to_numeric(plddt["pLDDT_SD"], errors="coerce")
    if plddt["UniProt_ID"].duplicated().any():
        raise ValueError("Human_Avg_pLDDTs.csv contains duplicate UniProt IDs")

    entanglement = pd.read_csv(args.entanglement, sep="|", low_memory=False)[["gene", "CCBond"]]
    entanglement["gene"] = entanglement["gene"].map(normalize_id)
    entanglement = (
        entanglement.groupby("gene", as_index=False)
        .agg({"CCBond": merge_ccbond})
        .rename(columns={"gene": "UniProt_ID", "CCBond": "NCLE"})
    )

    knots = pd.read_table(args.knots, low_memory=False)
    knot_ids = set(
        knots.loc[knots["Organism"].eq("Human"), "Uniprot"]
        .astype(str)
        .str.strip()
        .str.split("-")
        .str[0]
```

```
    )
    proteome_lengths = parse_fasta(args.proteome_fasta)
    membrane_ids = set(parse_fasta(args.membrane_fasta))
    secreted_ids = parse_id_source(args.secreted_source)

    monomer_ids: set[str] = set()
    for path in args.monomer_fasta:
        monomer_ids.update(parse_fasta(path))

    master = mcshane.merge(plddt, on="UniProt_ID", how="inner", validate="one_to_one")
    master = master.loc[master["Average_pLDDT"] >= 70].copy()
    master = master.merge(entanglement, on="UniProt_ID", how="left", validate="one_to_one")
    master["Knotted"] = master["UniProt_ID"].isin(knot_ids).astype(int)
    master["FASTA_available"] = master["UniProt_ID"].isin(proteome_lengths).astype(int)
    master["Protein_length"] = master["UniProt_ID"].map(proteome_lengths)
    master["Membrane"] = master["UniProt_ID"].isin(membrane_ids).astype(int)
    master["Secreted"] = master["UniProt_ID"].isin(secreted_ids).astype(int)
    master["Known_monomer_annotation"] = np.where(
        master["UniProt_ID"].isin(monomer_ids), "Known_monomer", "Unclassified"
    )
    master["Entanglement_status"] = master["NCLE"].map(
        {1.0: "Native_entangled", 0.0: "Not_entangled"}
    ).fillna("Excluded_covalent_lasso")

    id_sets = {
        "knots": knot_ids,
        "membrane": membrane_ids,
        "secreted": secreted_ids,
        "known_monomer": monomer_ids,
    }
    return master, id_sets, proteome_lengths


def build_cohort(master: pd.DataFrame, profiles: list[str], name: str) -> tuple[pd.DataFrame,
list[dict[str, object]]]:
    flow: list[dict[str, object]] = []
    data = master.loc[master["Degradation_profile"].isin(profiles)].copy()

    def record(step: str) -> None:
        counts = data["Degradation_profile"].value_counts()
        flow.append(
            {
                "Analysis": name,
                "Step": step,
                "Total": int(len(data)),
                "ED": int(counts.get("ED", 0)),
                "NED": int(counts.get("NED", 0)),
                "UN": int(counts.get("UN", 0)),
            }
        )

    record("pLDDT >=70 and selected degradation categories")
    data = data.loc[data["NCLE"].notna()].copy()
    record("Usable binary NCLE status")
    data = data.loc[data["Knotted"].eq(0)].copy()
    record("Exclude knotted proteins")
    data = data.loc[data["FASTA_available"].eq(1)].copy()
    record("Require FASTA sequence")
    data = data.loc[data["Membrane"].eq(0)].copy()
    record("Exclude membrane proteins")
    data = data.loc[data["Secreted"].eq(0)].copy()
    record("Exclude secreted proteins")

    data["NED_binary"] = data["Degradation_profile"].eq("NED").astype(int)
    data["z_Protein_length"] = (
        data["Protein_length"] - data["Protein_length"].mean()
    ) / data["Protein_length"].std(ddof=1)
    data["Analysis_group"] = (
        data["Degradation_profile"]
        + "_"
        + data["NCLE"].map({1.0: "Entangled", 0.0: "NotEntangled"})
    )
    data = data.sort_values("UniProt_ID").reset_index(drop=True)
    data.insert(0, "Index", np.arange(1, len(data) + 1))
    return data, flow
```

```
def crude_analysis(data: pd.DataFrame, analysis: str) -> tuple[dict[str, object], pd.DataFrame]:
    a = int(((data["NCLE"] == 1) & (data["NED_binary"] == 1)).sum())
    b = int(((data["NCLE"] == 1) & (data["NED_binary"] == 0)).sum())
    c = int(((data["NCLE"] == 0) & (data["NED_binary"] == 1)).sum())
    d = int(((data["NCLE"] == 0) & (data["NED_binary"] == 0)).sum())
    table = np.array([[a, b], [c, d]])
    odds_ratio, fisher_p = stats.fisher_exact(table, alternative="two-sided")
    log_se = np.sqrt(1 / a + 1 / b + 1 / c + 1 / d)
    lower, upper = np.exp(np.log(odds_ratio) + np.array([-1.96, 1.96]) * log_se)
    result = {
        "Analysis": analysis,
        "Method": "Crude Fisher exact",
        "Term": "NCLE",
        "N": int(len(data)),
        "Coefficient": float(np.log(odds_ratio)),
        "Odds_ratio": float(odds_ratio),
        "CI_lower": float(lower),
        "CI_upper": float(upper),
        "P_value": float(fisher_p),
    }
    counts = pd.DataFrame(
        [
            {"Analysis": analysis, "NCLE_status": "Entangled", "NED": a, "Non_NED": b, "Total": a + b},
            {"Analysis": analysis, "NCLE_status": "Not_entangled", "NED": c, "Non_NED": d, "Total": c + d},
        ]
    )
    counts["NED_frequency"] = counts["NED"] / counts["Total"]
    return result, counts


def logistic_analysis(data: pd.DataFrame, analysis: str, method: str, terms: list[str]) -> list[dict[str,
object]]:
    clean = data[["NED_binary", *terms]].dropna().copy()
    term_names = ["Intercept", *terms]
    x = np.column_stack([np.ones(len(clean)), clean[terms].to_numpy(dtype=float)])
    y = clean["NED_binary"].to_numpy(dtype=float)
    beta = np.zeros(x.shape[1], dtype=float)
    for _ in range(100):
        eta = np.clip(x @ beta, -35, 35)
        probability = 1.0 / (1.0 + np.exp(-eta))
        weights = np.clip(probability * (1.0 - probability), 1e-12, None)
        information = x.T @ (weights[:, None] * x)
        score = x.T @ (y - probability)
        step = np.linalg.solve(information, score)
        beta_next = beta + step
        if np.max(np.abs(step)) < 1e-10:
            beta = beta_next
            break
        beta = beta_next
    else:
        raise RuntimeError(f"Logistic regression did not converge for {analysis}: {method}")

    eta = np.clip(x @ beta, -35, 35)
    probability = 1.0 / (1.0 + np.exp(-eta))
    weights = np.clip(probability * (1.0 - probability), 1e-12, None)
    covariance = np.linalg.inv(x.T @ (weights[:, None] * x))
    standard_errors = np.sqrt(np.diag(covariance))
    coefficients = dict(zip(term_names, beta))
    errors = dict(zip(term_names, standard_errors))
    rows: list[dict[str, object]] = []
    for term in terms:
        coefficient = float(coefficients[term])
        standard_error = float(errors[term])
        lower_coefficient = coefficient - 1.96 * standard_error
        upper_coefficient = coefficient + 1.96 * standard_error
        rows.append(
            {
                "Analysis": analysis,
                "Method": method,
                "Term": term,
                "N": int(len(clean)),
                "Coefficient": coefficient,
                "Odds_ratio": float(np.exp(coefficient)),
```

```
                "CI_lower": float(np.exp(lower_coefficient)),
                "CI_upper": float(np.exp(upper_coefficient)),
                "P_value": float(2 * stats.norm.sf(abs(coefficient / standard_error))),
            }
        )
    return rows


def add_complex_annotation(data: pd.DataFrame, path: Path) -> pd.DataFrame:
    annotation = pd.read_csv(path)
    expected = {"UniProt_ID", "Complex_member"}
    if not expected.issubset(annotation.columns):
        raise ValueError(f"Complex annotation must contain columns: {sorted(expected)}")
    annotation = annotation[["UniProt_ID", "Complex_member"]].copy()
    annotation["UniProt_ID"] = annotation["UniProt_ID"].map(normalize_id)
    annotation["Complex_member"] = pd.to_numeric(annotation["Complex_member"], errors="coerce")
    if annotation["UniProt_ID"].duplicated().any():
        raise ValueError("Complex annotation contains duplicate UniProt IDs")
    if not set(annotation["Complex_member"].dropna().unique()).issubset({0, 1}):
        raise ValueError("Complex_member must be 0, 1, or missing")
    return data.merge(annotation, on="UniProt_ID", how="left", validate="one_to_one")


def main() -> None:
    args = parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=True)
    master, id_sets, proteome_lengths = load_sources(args)

    primary, flow_primary = build_cohort(master, ["ED", "NED"], "Primary: NED vs ED")
    sensitivity, flow_sensitivity = build_cohort(
        master, ["ED", "NED", "UN"], "Sensitivity: NED vs ED + UN"
    )

    results: list[dict[str, object]] = []
    count_tables: list[pd.DataFrame] = []
    for label, data in [
        ("Primary: NED vs ED", primary),
        ("Sensitivity: NED vs ED + UN", sensitivity),
    ]:
        crude, counts = crude_analysis(data, label)
        results.append(crude)
        count_tables.append(counts)
        results.extend(
            logistic_analysis(
                data,
                label,
                "Model 1: NED ~ NCLE + z(length)",
                ["NCLE", "z_Protein_length"],
            )
        )

    complex_status = "Not run: accurate binary complex-membership annotation was not supplied."
    if args.complex_annotation:
        complex_status = "Run using the supplied Complex_member annotation."
        updated: list[pd.DataFrame] = []
        for label, data in [
            ("Primary: NED vs ED", primary),
            ("Sensitivity: NED vs ED + UN", sensitivity),
        ]:
            annotated = add_complex_annotation(data, args.complex_annotation)
            results.extend(
                logistic_analysis(
                    annotated,
                    label,
                    "Model 2: NED ~ NCLE + complex + z(length)",
                    ["NCLE", "Complex_member", "z_Protein_length"],
                )
            )
            updated.append(annotated)
        primary, sensitivity = updated

    results_df = pd.DataFrame(results)
    counts_df = pd.concat(count_tables, ignore_index=True)
    flow_df = pd.DataFrame(flow_primary + flow_sensitivity)
```

```
    ordered_columns = [
        "Index",
        "UniProt_ID",
        "Degradation_profile",
        "NED_binary",
        "NCLE",
        "Entanglement_status",
        "Average_pLDDT",
        "pLDDT_SD",
        "Protein_length",
        "z_Protein_length",
        "Knotted",
        "FASTA_available",
        "Membrane",
        "Secreted",
        "Known_monomer_annotation",
        "Analysis_group",
    ]
    if "Complex_member" in primary.columns:
        ordered_columns.append("Complex_member")
    primary[ordered_columns].to_csv(args.output_dir / "primary_cohort.csv", index=False)
    sensitivity[ordered_columns].to_csv(args.output_dir / "sensitivity_cohort.csv", index=False)
    results_df.to_csv(args.output_dir / "association_results.csv", index=False)
    counts_df.to_csv(args.output_dir / "contingency_counts.csv", index=False)
    flow_df.to_csv(args.output_dir / "cohort_flow.csv", index=False)

    metadata = {
        "primary_n": int(len(primary)),
        "sensitivity_n": int(len(sensitivity)),
        "primary_group_counts": primary["Analysis_group"].value_counts().sort_index().to_dict(),
        "sensitivity_profile_ncle_counts": {
            f"{profile}|NCLE={int(ncle)}": int(count)
            for (profile, ncle), count in sensitivity.groupby(
                ["Degradation_profile", "NCLE"]
            ).size().items()
        },
        "secreted_source_ids": len(id_sets["secreted"]),
        "known_monomer_ids": len(id_sets["known_monomer"]),
        "complex_model_status": complex_status,
        "notes": [
            "MobiDB disorder was not used to select either statistical cohort.",
            "NCLE=1 denotes native non-covalent entanglement; NCLE=0 denotes no detected entanglement.",
            "Proteins with covalent lasso records are excluded from binary NCLE analyses.",
            "Known_monomer_annotation=Unclassified must not be interpreted as complex membership.",
        ],
    }
    (args.output_dir / "analysis_metadata.json").write_text(json.dumps(metadata, indent=2, default=str))

    print(json.dumps(metadata, indent=2, default=str))
    print("\nAssociation results:")
    print(results_df.to_string(index=False))


if __name__ == "__main__":
    main()
```

Appendix 2. reproduce_results.py

Location: scripts/reproduce_results.py
Provenance: New, validated reproduction runner

Reads the final workbook protein rows, validates coding and eligibility flags, recomputes all 12 reported result rows, and compares them with the workbook. Also fits Model 2 with a separate SciPy optimizer and writes missing-annotation, restricted-sample, and provisional-call diagnostics.

```
#!/usr/bin/env python3
"""Recompute human results from frozen protein rows, not stored ORs.

This is a new reproduction runner. statistical_analysis.py is the recovered
original analysis code. This runner never refreshes online annotations.
"""
```

```
import argparse
import hashlib
import importlib.metadata
import json
import sys
from datetime import datetime, timezone
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import minimize
from scipy.special import expit
from scipy.stats import fisher_exact, norm

from statistical_analysis import crude_analysis, logistic_analysis

ROOT = Path(__file__).resolve().parents[1]
LABELS = {
    "Primary": "Primary: NED vs ED",
    "Sensitivity": "Sensitivity: NED vs ED + UN",
}
M1 = "Model 1: NED ~ NCLE + z(length)"
M2 = "Model 2: NED ~ NCLE + complex + z(length)"
METRICS = ["N", "Coefficient", "Odds_ratio", "CI_lower", "CI_upper", "P_value"]


def sha256(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def validate_cohort(data, primary):
    if data["UniProt_ID"].isna().any() or data["UniProt_ID"].duplicated().any():
        raise ValueError("Missing or repeated UniProt ID")
    allowed = {"ED", "NED"} if primary else {"ED", "NED", "UN"}
    if not set(data["Degradation_profile"]).issubset(allowed):
        raise ValueError("Unexpected degradation category")
    for col in ["NCLE", "NED_binary"]:
        if data[col].isna().any() or not set(data[col]).issubset({0, 1}):
            raise ValueError(f"{col} must be complete and binary")
    expected_y = data["Degradation_profile"].eq("NED").astype(int)
    if not data["NED_binary"].eq(expected_y).all():
        raise ValueError("Outcome does not agree with degradation profile")
    length = pd.to_numeric(data["Protein_length"], errors="raise")
    if length.isna().any() or not length.gt(0).all() or length.std(ddof=1) <= 0:
        raise ValueError("Invalid protein lengths")
    if not data["Average_pLDDT"].ge(70).all():
        raise ValueError("A protein fails the inclusive pLDDT criterion")
    for col in ["Knotted", "Membrane", "Secreted"]:
        if not data[col].eq(0).all():
            raise ValueError(f"Excluded {col} protein in cohort")
    if not data["FASTA_available"].eq(1).all():
        raise ValueError("A FASTA sequence is missing")
    if not set(data["Complex_member"].dropna()).issubset({0, 1}):
        raise ValueError("Invalid complex-membership value")
    z = (length - length.mean()) / length.std(ddof=1)
    if not np.allclose(z, data["z_Protein_length"], atol=1e-12, rtol=1e-12):
        raise ValueError("Stored standardized lengths are inconsistent")
    # Standardize once within each full cohort, before Model 2 exclusions.
    data = data.copy()
    data["z_Protein_length"] = z
    return data


def independent_fit(data, label):
    """Independent MLE fit using a different optimizer and analytic Hessian."""
    terms = ["NCLE", "Complex_member", "z_Protein_length"]
    clean = data.dropna(subset=["NED_binary", *terms])
    x = np.column_stack([np.ones(len(clean)), clean[terms].to_numpy(float)])
    y = clean["NED_binary"].to_numpy(float)
    if np.linalg.matrix_rank(x) != x.shape[1]:
        raise ValueError("Model 2 design matrix is not full rank")

    def objective(beta):
        eta = x @ beta
        return np.logaddexp(0, eta).sum() - y @ eta
```

```
    def gradient(beta):
        return x.T @ (expit(x @ beta) - y)

    def hessian(beta):
        p = expit(x @ beta)
        return x.T @ ((p * (1 - p))[:, None] * x)

    fit = minimize(objective, np.zeros(x.shape[1]), jac=gradient,
                   hess=hessian, method="trust-exact",
                   options={"gtol": 1e-9, "maxiter": 500})
    if np.max(np.abs(gradient(fit.x))) >= 1e-6:
        raise RuntimeError(f"Independent fit did not converge: {fit.message}")
    se = np.sqrt(np.diag(np.linalg.inv(hessian(fit.x))))
    rows = []
    for i, term in enumerate(terms, 1):
        b = fit.x[i]
        rows.append(dict(Analysis=label, Method=M2, Term=term, N=len(clean),
                         Coefficient=b, Odds_ratio=np.exp(b),
                         CI_lower=np.exp(b - 1.96 * se[i]),
                         CI_upper=np.exp(b + 1.96 * se[i]),
                         P_value=2 * norm.sf(abs(b / se[i]))))
    return rows


def compare_results(actual, reference, description):
    keys = ["Analysis", "Method", "Term"]
    if actual.duplicated(keys).any() or reference.duplicated(keys).any():
        raise ValueError("Repeated result key")
    joined = actual.merge(reference, on=keys, how="outer",
                          suffixes=("_actual", "_reference"),
                          indicator=True, validate="one_to_one")
    rows = []
    for _, row in joined.iterrows():
        for metric in METRICS:
            a, b = row[f"{metric}_actual"], row[f"{metric}_reference"]
            match = row["_merge"] == "both" and np.isfinite(a) and np.isfinite(b)
            match = match and (a == b if metric == "N" else
                               np.isclose(a, b, atol=1e-8, rtol=1e-8))
            rows.append({**{k: row[k] for k in keys}, "Comparison": description,
                         "Metric": metric, "Actual": a, "Reference": b,
                         "Absolute_difference": abs(a - b), "Pass": bool(match)})
    return pd.DataFrame(rows)


def binary_diagnostic(data, exposed, comparison, label):
    y = data["NED_binary"].eq(1)
    a, b = int((exposed & y).sum()), int((exposed & ~y).sum())
    c, d = int((~exposed & y).sum()), int((~exposed & ~y).sum())
    odds, p = fisher_exact([[a, b], [c, d]], alternative="two-sided")
    return dict(Analysis=label, Comparison=comparison, Exposed_NED=a,
                Exposed_non_NED=b, Reference_NED=c, Reference_non_NED=d,
                Odds_ratio=float(odds), P_value=float(p))


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--reference", type=Path,
                        default=ROOT / "data/reference_human_analysis.xlsx")
    parser.add_argument("--out", type=Path, default=ROOT / "results/frozen_replay")
    args = parser.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    cohorts = {}
    for name in LABELS:
        d = pd.read_excel(args.reference, sheet_name=f"{name} Cohort")
        cohorts[name] = validate_cohort(d, name == "Primary")
    primary_ids = set(cohorts["Primary"]["UniProt_ID"])
    sensitivity_edned = cohorts["Sensitivity"].query("Degradation_profile != 'UN'")
    if primary_ids != set(sensitivity_edned["UniProt_ID"]):
        raise ValueError("Primary and sensitivity ED/NED memberships differ")

    results, counts, restricted, exclusions, diagnostics = [], [], [], [], []
    independent, status_counts = [], []
    for name, data in cohorts.items():
        label = LABELS[name]
```

```
        crude, table = crude_analysis(data, label)
        results.append(crude)
        counts.append(table)
        results.extend(logistic_analysis(data, label, M1, ["NCLE", "z_Protein_length"]))
        results.extend(logistic_analysis(data, label, M2,
                                         ["NCLE", "Complex_member", "z_Protein_length"]))
        complete = data.dropna(subset=["Complex_member"])
        restricted.extend(logistic_analysis(complete, label,
                          "Restricted Model 1: NED ~ NCLE + z(length)",
                          ["NCLE", "z_Protein_length"]))
        provisional = data["Manual_review_decision"].eq("Complex_or_oligomer")
        exclusions.extend(logistic_analysis(data.loc[~provisional], label,
                          "Model 2 excluding provisional positive calls",
                          ["NCLE", "Complex_member", "z_Protein_length"]))
        diagnostics.append(binary_diagnostic(complete, complete["Complex_member"].eq(1),
                           "Complex member versus explicit monomer", label))
        diagnostics.append(binary_diagnostic(data, data["Complex_member"].notna(),
                           "Usable versus missing complex annotation", label))
        for status, group in data.groupby("Oligomer_status", dropna=False):
            status_counts.append(dict(Analysis=label, Status=status, N=len(group),
                                      NED=int(group["NED_binary"].sum())))
        independent.extend(independent_fit(data, label))
        data.to_csv(args.out / f"{name.lower()}_cohort.csv", index=False)

    actual = pd.DataFrame(results)
    expected = pd.read_excel(args.reference, sheet_name="Association Results")
    comparison = compare_results(actual, expected, "Reported workbook results")
    independent_comparison = compare_results(pd.DataFrame(independent),
                             actual.loc[actual["Method"].eq(M2)],
                             "Independent optimizer versus original solver")
    outputs = {
        "association_results.csv": actual,
        "contingency_counts.csv": pd.concat(counts, ignore_index=True),
        "restricted_model1.csv": pd.DataFrame(restricted),
        "provisional_exclusion_results.csv": pd.DataFrame(exclusions),
        "model2_fisher_diagnostics.csv": pd.DataFrame(diagnostics),
        "annotation_status_counts.csv": pd.DataFrame(status_counts),
        "reference_comparison.csv": comparison,
        "independent_optimizer_comparison.csv": independent_comparison,
    }
    for name, data in outputs.items():
        data.to_csv(args.out / name, index=False)
    passed = comparison["Pass"].all() and independent_comparison["Pass"].all()
    manifest = {
        "run_utc": datetime.now(timezone.utc).isoformat(), "python": sys.version,
        "reference_sha256": sha256(args.reference),
        "original_analysis_sha256": sha256(Path(__file__).with_name("statistical_analysis.py")),
        "runner_sha256": sha256(__file__), "numerical_comparison_passed": bool(passed),
        "primary_n": len(cohorts["Primary"]),
        "sensitivity_n": len(cohorts["Sensitivity"]),
        "versions": {p: importlib.metadata.version(p)
                     for p in ["numpy", "pandas", "scipy", "openpyxl"]},
        "scope": "Frozen-cohort numerical replay; no online annotation refresh",
        "confidence_interval_multiplier": 1.96,
    }
    (args.out / "run_manifest.json").write_text(json.dumps(manifest, indent=2))
    print(actual.loc[actual["Term"].eq("NCLE")].to_string(index=False))
    print(f"\nNumerical reproduction: {'PASS' if passed else 'FAIL'}")
    print(f"Results: {args.out.resolve()}")
    if not passed:
        raise SystemExit(1)


if __name__ == "__main__":
    main()
```

Appendix 3. export_complex_annotations.py

Location: scripts/export_complex_annotations.py
Provenance: New, validated reconstruction utility

Reconstructs the recorded final complex covariate from saved initial labels and recorded manual decisions. Exports the binary calls, UniProt evidence, and review decisions. It does not recreate the unrecovered automatic text-classification implementation or independently confirm a biological label.

```
#!/usr/bin/env python3
"""Export evidence and replay recorded review decisions from the final workbook.

This does not recreate the unavailable original text-classification code or
independently validate its biological decisions. It reconstructs final calls
from frozen automated calls plus the recorded first-pass review decisions.
"""
import argparse
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--reference", type=Path,
                        default=ROOT / "data/reference_human_analysis.xlsx")
    parser.add_argument("--out", type=Path, default=ROOT / "data")
    args = parser.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    evidence = pd.read_excel(args.reference, sheet_name="UniProt Evidence")
    decisions = pd.read_excel(args.reference, sheet_name="Manual Review")
    if evidence["UniProt_ID"].isna().any() or evidence["UniProt_ID"].duplicated().any():
        raise ValueError("Evidence requires one row per protein")
    if decisions["UniProt_ID"].isna().any() or decisions["UniProt_ID"].duplicated().any():
        raise ValueError("Review decisions contain missing or repeated IDs")
    ambiguous_ids = set(evidence.loc[evidence["Automated_Oligomer_status"].eq("Ambiguous"),
                                     "UniProt_ID"])
    if set(decisions["UniProt_ID"]) != ambiguous_ids:
        raise ValueError("Review worksheet does not cover exactly the ambiguous records")
    decision_map = decisions.set_index("UniProt_ID")["Manual_review_decision"]
    if not set(decision_map.dropna()).issubset({"Complex_or_oligomer", "Unresolved"}):
        raise ValueError("Unexpected recorded review decision")
    recorded = evidence["UniProt_ID"].map(decision_map)
    if not recorded.fillna("").eq(evidence["Manual_review_decision"].fillna("")).all():
        raise ValueError("Evidence and review worksheets disagree")
    calls = evidence["Automated_Complex_member"].copy()
    # A workflow-completion flag never supplies a biological classification.
    positive = recorded.eq("Complex_or_oligomer")
    unresolved = recorded.eq("Unresolved")
    calls.loc[positive] = 1
    calls.loc[unresolved] = np.nan
    if not np.allclose(calls, evidence["Complex_member"], equal_nan=True):
        raise ValueError("Recorded review decisions do not reconstruct final calls")
    annotation = pd.DataFrame({"UniProt_ID": evidence["UniProt_ID"],
                               "Complex_member": calls})
    annotation.to_csv(args.out / "complex_annotations.csv", index=False)
    evidence.to_csv(args.out / "uniprot_evidence_snapshot.csv", index=False)
    decisions.to_csv(args.out / "complex_review_decisions.csv", index=False)
    print(f"Evidence proteins: {len(evidence)}")
    print(f"Initially ambiguous: {len(decisions)}")
    print(f"Provisional positive calls: {int(positive.sum())}")
    print(f"Reviewed unresolved: {int(unresolved.sum())}")
    print("Recorded-decision reconstruction: PASS")


if __name__ == "__main__":
    main()
```

Appendix 4. audit_raw_sources.py

Location: scripts/audit_raw_sources.py
Provenance: New, validated source audit

Checks source uniqueness and the raw integration/filtering counts. Without the approved secreted export it stops at the pre-secreted cohorts, explicitly marks that final check as not run, and never manufactures secreted calls from eligible IDs. With that export it also compares final IDs and key fields.

```
#!/usr/bin/env python3
"""Replay raw cohort filters and compare them with the final reference.

Without --secreted-source, validation stops before secreted exclusion. The
final eligible IDs are never used to manufacture a secreted annotation list.
"""
import argparse
import hashlib
import json
import re
import tempfile
from pathlib import Path
from types import SimpleNamespace

import numpy as np
import pandas as pd

from statistical_analysis import load_sources, build_cohort

ROOT = Path(__file__).resolve().parents[1]


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--sources", type=Path, default=ROOT / "raw_sources")
    parser.add_argument("--reference", type=Path,
                        default=ROOT / "data/reference_human_analysis.xlsx")
    parser.add_argument("--secreted-source", type=Path)
    parser.add_argument("--out", type=Path, default=ROOT / "results/raw_source_audit")
    args = parser.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    sources = {
        "mcshane": args.sources / "McShane_human_mmc4.xlsx",
        "entanglement": args.sources / "Human_AF_combined_20250614.csv",
        "plddt": args.sources / "Human_Avg_pLDDTs.csv",
        "knots": args.sources / "AF4_knots.dat",
        "proteome_fasta": args.sources / "UP000005640_9606.fasta",
        "membrane_fasta": args.sources / "uniprotkb_human_keywords_membrane_2025_03_19.fasta",
    }
    missing = [str(p) for p in sources.values() if not p.is_file()]
    if missing:
        raise FileNotFoundError("Missing raw source files: " + ", ".join(missing))
    for key in ["proteome_fasta", "membrane_fasta"]:
        ids = []
        with sources[key].open() as handle:
            for line in handle:
                if line.startswith(">"):
                    pieces = line[1:].strip().split("|")
                    ids.append(pieces[1] if len(pieces) >= 3 else pieces[0].split()[0])
        if len(ids) != len(set(ids)):
            raise ValueError(f"Duplicate accessions in {key}; do not concatenate sequences")
    full = pd.read_excel(sources["mcshane"], sheet_name="FinalTable_RPE_")
    if full["Protein IDs (Uniprot)"].isna().any() or full["Protein IDs (Uniprot)"].duplicated().any():
        raise ValueError("McShane identifiers are missing or duplicated")
    ent = pd.read_csv(sources["entanglement"], sep="|", usecols=["gene", "CCBond"])
    tokens = set(ent["CCBond"].dropna().astype(str).str.strip())
    if tokens - {"True", "False", "1", "0", "1.0", "0.0"}:
        raise ValueError(f"Unknown CCBond tokens: {tokens}")
    with tempfile.TemporaryDirectory() as tmp:
        empty = Path(tmp) / "empty_for_presecreted_validation.json"
        empty.write_text("[]")
        ns = SimpleNamespace(**sources, secreted_source=args.secreted_source or empty,
                             monomer_fasta=[])
        master, sets, lengths = load_sources(ns)
```

```
    layout = pd.read_excel(args.reference, sheet_name="Cohort Flow", header=None)
    comparisons, membership = [], []
    for name, profiles in [("Primary", ["ED", "NED"]),
                           ("Sensitivity", ["ED", "NED", "UN"])]:
        label = "Primary: NED vs ED" if name == "Primary" else "Sensitivity: NED vs ED + UN"
        data, flow = build_cohort(master, profiles, label)
        if args.secreted_source is None:
            flow = flow[:-1]
            data["Secreted"] = np.nan
        for entry in flow:
            ref = layout.loc[layout[0].eq(label) & layout[1].eq(entry["Step"])]
            if len(ref) != 1:
                raise ValueError("Reference flow step is missing or repeated")
            for i, metric in enumerate(["Total", "ED", "NED", "UN"], 2):
                expected = int(ref.iloc[0, i])
                comparisons.append(dict(Analysis=label, Step=entry["Step"], Metric=metric,
                                        Actual=entry[metric], Reference=expected,
                                        Pass=entry[metric] == expected))
        prefix = "rebuilt" if args.secreted_source else "pre_secreted"
        data.to_csv(args.out / f"{prefix}_{name.lower()}_cohort.csv", index=False)
        if args.secreted_source:
            ref = pd.read_excel(args.reference, sheet_name=f"{name} Cohort")
            joined = data.merge(ref, on="UniProt_ID", how="outer", indicator=True,
                                suffixes=("_rebuilt", "_reference"), validate="one_to_one")
            for _, row in joined.iterrows():
                same = row["_merge"] == "both"
                for field in ["NCLE", "NED_binary", "Protein_length", "Average_pLDDT"]:
                    same = same and np.isclose(row[field + "_rebuilt"], row[field + "_reference"],
                                               atol=1e-10, rtol=1e-10)
                membership.append(dict(Analysis=label, UniProt_ID=row["UniProt_ID"],
                                       Membership=row["_merge"], Pass=bool(same)))
    result = pd.DataFrame(comparisons)
    result.to_csv(args.out / "cohort_flow_comparison.csv", index=False)
    if membership:
        pd.DataFrame(membership).to_csv(args.out / "final_membership_comparison.csv", index=False)
    # Preserve the historical normalization, and expose its broader scope.
    knot = pd.read_table(sources["knots"])
    human = knot.loc[knot["Organism"].eq("Human"), "Uniprot"].astype(str).str.strip()
    broad = set(human.str.split("-").str[0])
    strict = {re.sub(r"-F\d+$", "", x) for x in human}
    qc_ids = set(pd.read_csv(sources["plddt"]).query("`<pLDDT>` >= 70")["gene"])
    meta = dict(mcshane_rows=len(full), degradation_counts=full["Degradation
profile"].value_counts().to_dict(),
                entanglement_rows=len(ent), entanglement_proteins=ent["gene"].nunique(),
                historical_knotted_in_plddt_master=len(qc_ids & broad),
                strict_fragment_knotted_in_plddt_master=len(qc_ids & strict),
                broader_knot_normalization_extra_ids=sorted((qc_ids & broad) - strict),
                secreted_source_supplied=args.secreted_source is not None,
                final_cohort_source_validation="RUN" if args.secreted_source else "NOT RUN",
                source_sha256={k: hashlib.sha256(p.read_bytes()).hexdigest() for k, p in sources.items()})
    (args.out / "raw_source_manifest.json").write_text(json.dumps(meta, indent=2))
    print(result.to_string(index=False))
    print("Final secreted exclusion: " + ("checked" if args.secreted_source else
          "NOT independently replayed; supply the approved historical keyword export"))
    if not result["Pass"].all() or (membership and not all(x["Pass"] for x in membership)):
        raise SystemExit(1)


if __name__ == "__main__":
    main()
```

Appendix 5. audit_disorder.py

Location: scripts/audit_disorder.py
Provenance: New, validated processed-data audit

Checks fraction = clipped disorder count / sequence length, interval unions, query statuses, and fraction coverage in the statistics cohorts. Exports coordinate warnings and annotates cohorts without filtering their membership by disorder availability.

```
#!/usr/bin/env python3
```

```
"""Audit the frozen processed MobiDB export and its statistical-cohort coverage.

The export is processed output, not raw API JSON. Coordinate clipping is
reported explicitly; it does not establish sequence-version compatibility.
"""
import argparse
import json
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]


def intervals(value):
    if pd.isna(value) or str(value).strip() in {"", "-"}:
        return []
    parsed = []
    for token in str(value).split(";"):
        start, end = map(int, token.strip().split("-"))
        if start > end:
            raise ValueError(f"Reversed interval: {token}")
        parsed.append((start, end))
    parsed.sort()
    merged = []
    for start, end in parsed:
        if merged and start <= merged[-1][1] + 1:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--master", type=Path, default=ROOT / "data/processed_MobiDB_master.csv")
    parser.add_argument("--reference", type=Path, default=ROOT / "data/reference_human_analysis.xlsx")
    parser.add_argument("--out", type=Path, default=ROOT / "results/disorder_audit")
    args = parser.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    d = pd.read_csv(args.master)
    if d["UniProt_ID"].isna().any() or d["UniProt_ID"].duplicated().any():
        raise ValueError("Master identifiers must be present and unique")
    fraction = pd.to_numeric(d["Fraction_IDP_IDR"], errors="raise")
    length = pd.to_numeric(d["MobiDB_length"], errors="raise")
    count = pd.to_numeric(d["Disorder_residues"], errors="raise")
    available = fraction.notna()
    if not fraction.loc[available].between(0, 1).all():
        raise ValueError("Out-of-range fraction")
    if not length.loc[available].gt(0).all() or not count.loc[available].ge(0).all():
        raise ValueError("Invalid denominator or disorder count")
    if not np.allclose(fraction.loc[available], (count / length).loc[available], atol=1e-8, rtol=0):
        raise ValueError("Fraction disagrees with count divided by length")
    ok = d["Disorder_query_status"].eq("ok")
    if not ok.eq(available).all():
        raise ValueError("Status and fraction availability disagree")
    issues = []
    for _, row in d.loc[available].iterrows():
        regions = intervals(row["Disorder_regions"])
        full = sum(end - start + 1 for start, end in regions)
        size = int(row["MobiDB_length"])
        clipped = sum(max(0, min(size, end) - max(1, start) + 1) for start, end in regions)
        if clipped != int(row["Disorder_residues"]):
            raise ValueError(f"Clipped count does not reproduce stored count: {row['UniProt_ID']}")
        if full != clipped:
            issues.append(dict(UniProt_ID=row["UniProt_ID"], Full_interval_count=full,
                               Clipped_count=clipped, MobiDB_length=size,
                               Disorder_regions=row["Disorder_regions"]))
    pd.DataFrame(issues).to_csv(args.out / "interval_coordinate_warnings.csv", index=False)
    rows = []
    for name in ["Primary", "Sensitivity"]:
        cohort = pd.read_excel(args.reference, sheet_name=f"{name} Cohort")
        merged = cohort.merge(d[["UniProt_ID", "Fraction_IDP_IDR"]],
                              how="left", on="UniProt_ID", validate="one_to_one")
```

```
        merged["Fraction_available"] = merged["Fraction_IDP_IDR"].notna()
        for group, sub in merged.groupby("Analysis_group"):
            n, found = len(sub), int(sub["Fraction_available"].sum())
            rows.append(dict(Analysis=name, Group=group, Total=n, Fraction_available=found,
                             Fraction_missing=n-found, Missing_percent=100*(n-found)/n))
        # Coverage output has no disorder cutoff and is not the statistical sample.
        merged.to_csv(args.out / f"{name.lower()}_cohort_with_disorder.csv", index=False)
    pd.DataFrame(rows).to_csv(args.out / "disorder_coverage.csv", index=False)
    summary = dict(master_n=len(d), status_counts=d["Disorder_query_status"].value_counts().to_dict(),
                   fraction_available=int(available.sum()), interval_coordinate_warnings=len(issues),
                   source_kind="Processed export; raw API responses are not included")
    (args.out / "disorder_audit_summary.json").write_text(json.dumps(summary, indent=2))
    print(json.dumps(summary, indent=2))
    print(pd.DataFrame(rows).to_string(index=False))


if __name__ == "__main__":
    main()
```

Appendix 6. prepare_mobidb_ids.py

Location: scripts/prepare_mobidb_ids.py
Provenance: New, validated input utility

Uses Jiang's dated average-pLDDT file with an inclusive >=70 threshold to generate the 13,690-ID broad query list and a five-ID test list. It does not contact MobiDB.

```
#!/usr/bin/env python3
"""Export query IDs using Jiang's historical average-pLDDT file.

This reconstructed utility prepares inputs; it does not query MobiDB.
The inclusive cutoff reproduces the 13,690-protein broad query list.
"""
import argparse
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[1]


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--plddt", type=Path,
                        default=ROOT / "raw_sources/Human_Avg_pLDDTs.csv")
    parser.add_argument("--out", type=Path, default=ROOT / "results/query_inputs")
    parser.add_argument("--cutoff", type=float, default=70)
    args = parser.parse_args()
    data = pd.read_csv(args.plddt)
    if data["gene"].isna().any() or data["gene"].duplicated().any():
        raise ValueError("Missing or repeated pLDDT accession")
    scores = pd.to_numeric(data["<pLDDT>"], errors="raise")
    selected = data.loc[scores.ge(args.cutoff)].copy()
    ids = selected["gene"].astype(str).str.strip()
    if ids.eq("").any() or ids.duplicated().any():
        raise ValueError("Invalid normalized query ID")
    args.out.mkdir(parents=True, exist_ok=True)
    cutoff_tag = f"{args.cutoff:g}".replace(".", "p")
    ids.to_csv(args.out / f"uniprots_plddt_ge{cutoff_tag}.txt", index=False, header=False)
    ids.head(5).to_csv(args.out / "uniprots_test5.txt", index=False, header=False)
    selected.to_csv(args.out / "selected_plddt_source.csv", index=False)
    print(f"Average pLDDT >= {args.cutoff:g}: {len(ids):,} unique proteins")
    print(f"Query inputs: {args.out.resolve()}")


if __name__ == "__main__":
    main()
```

Appendix 7. query_mobidb_disorder_production.py

Location: scripts/query_mobidb_disorder_production.py
Provenance: Recovered archived distributed script; unchanged

Queries MobiDB and UniProt, unions all selected available disorder tracks, clips counts to UniProt sequence length, and writes explicit statuses and resumable TSV output. No live API query was performed during this reproduction. The archived copy does not establish any later laptop-specific IPv4 modification.

```
#!/usr/bin/env python3
"""Query MobiDB/UniProt and calculate the per-protein disorder fraction.

This production version preserves the disorder-track union used in Quyen's
sample script, while adding retries, explicit failure statuses, incremental
output, and resume support. Failed requests remain missing rather than being
misclassified as zero disorder.
"""

from __future__ import annotations

import argparse
import csv
import json
import os
import sys
import time
from collections import Counter, OrderedDict
from pathlib import Path
from typing import Iterable
from urllib.parse import quote

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry


DISORDER_KEYWORDS = [
    "curated-disorder-priority",
    "homology-disorder-priority",
    "derived-missing_residues-priority",
    "prediction-disorder-priority",
    "derived-mobile-th_90",
]

FIELDNAMES = [
    "uniprot",
    "disorder_regions",
    "num_disorder_residues",
    "length",
    "fraction_idp_idr",
    "query_status",
    "annotation_tracks",
]


def build_session() -> requests.Session:
    retry = Retry(
        total=5,
        connect=5,
        read=5,
        status=5,
        backoff_factor=1.0,
        status_forcelist=(429, 500, 502, 503, 504),
        allowed_methods=frozenset({"GET"}),
        respect_retry_after_header=True,
        raise_on_status=False,
    )
    session = requests.Session()
    session.headers.update(
        {
            "User-Agent": "protein-decay-disorder-table/1.0",
            "Accept": "application/json",
        }
```

```
    )
    adapter = HTTPAdapter(max_retries=retry)
    session.mount("https://", adapter)
    session.mount("http://", adapter)
    return session


def request_json(session: requests.Session, url: str, timeout: int = 45):
    try:
        response = session.get(url, timeout=timeout)
    except requests.RequestException as exc:
        return None, f"request_failed:{type(exc).__name__}"

    if response.status_code == 404:
        return None, "not_found"
    if not response.ok:
        return None, f"http_{response.status_code}"

    try:
        return response.json(), None
    except (json.JSONDecodeError, ValueError):
        return None, "invalid_json"


def normalize_mobidb_payload(payload):
    if isinstance(payload, dict):
        return payload
    if isinstance(payload, list) and len(payload) == 1 and isinstance(payload[0], dict):
        return payload[0]
    return None


def get_mobidb_regions(session: requests.Session, uniprot: str):
    url = f"https://mobidb.org/api/download?format=json&acc={quote(uniprot)}"
    payload, error = request_json(session, url)
    if error:
        return None, [], f"mobidb_{error}"

    result = normalize_mobidb_payload(payload)
    if result is None:
        return None, [], "mobidb_unexpected_payload"

    data_sets = []
    tracks = []
    for key in DISORDER_KEYWORDS:
        block = result.get(key)
        if not isinstance(block, dict):
            continue
        regions = block.get("regions")
        if not isinstance(regions, list):
            continue
        tracks.append(key)
        data_sets.append(regions)

    if not tracks:
        return None, [], "mobidb_no_selected_annotation"

    return data_sets, tracks, None


def get_uniprot_length(session: requests.Session, uniprot: str):
    url = f"https://rest.uniprot.org/uniprotkb/{quote(uniprot)}.json"
    payload, error = request_json(session, url)
    if error:
        return None, f"uniprot_{error}"
    try:
        length = int(payload["sequence"]["length"])
    except (KeyError, TypeError, ValueError):
        return None, "uniprot_missing_length"
    if length <= 0:
        return None, "uniprot_invalid_length"
    return length, None


def merge_intervals(data_sets: Iterable[Iterable]) -> tuple[tuple[int, int], ...]:
```

```
    intervals = []
    for data_set in data_sets:
        for region in data_set:
            if not isinstance(region, (list, tuple)) or len(region) < 2:
                continue
            try:
                start = int(region[0])
                end = int(region[1])
            except (TypeError, ValueError):
                continue
            if start > end:
                start, end = end, start
            intervals.append((start, end))

    if not intervals:
        return ()

    intervals.sort(key=lambda item: (item[0], item[1]))
    merged = [list(intervals[0])]
    for start, end in intervals[1:]:
        if start <= merged[-1][1] + 1:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return tuple((start, end) for start, end in merged)


def count_disordered_residues(regions, length: int) -> int:
    total = 0
    for start, end in regions:
        clipped_start = max(1, start)
        clipped_end = min(length, end)
        if clipped_end >= clipped_start:
            total += clipped_end - clipped_start + 1
    return total


def format_regions(regions) -> str:
    return ";".join(f"{start}-{end}" for start, end in regions) if regions else "-"


def query_one(session: requests.Session, uniprot: str) -> dict[str, str]:
    data_sets, tracks, mobidb_error = get_mobidb_regions(session, uniprot)
    length, length_error = get_uniprot_length(session, uniprot)

    record = {field: "" for field in FIELDNAMES}
    record["uniprot"] = uniprot
    record["annotation_tracks"] = ";".join(tracks)

    errors = [error for error in (mobidb_error, length_error) if error]
    if errors:
        if length is not None:
            record["length"] = str(length)
        record["query_status"] = "|".join(errors)
        return record

    regions = merge_intervals(data_sets)
    disorder_residues = count_disordered_residues(regions, length)
    fraction = disorder_residues / length
    if not 0.0 <= fraction <= 1.0:
        record["query_status"] = "invalid_fraction"
        return record

    record.update(
        {
            "disorder_regions": format_regions(regions),
            "num_disorder_residues": str(disorder_residues),
            "length": str(length),
            "fraction_idp_idr": f"{fraction:.8f}",
            "query_status": "ok",
        }
    )
    return record
```

```
def read_ids(path: Path) -> list[str]:
    seen = set()
    ids = []
    with path.open() as handle:
        for line in handle:
            accession = line.strip()
            if accession and accession not in seen:
                seen.add(accession)
                ids.append(accession)
    return ids


def read_existing(path: Path) -> OrderedDict[str, dict[str, str]]:
    records = OrderedDict()
    if not path.exists() or path.stat().st_size == 0:
        return records
    with path.open(newline="") as handle:
        reader = csv.DictReader(handle, delimiter="\t")
        if reader.fieldnames != FIELDNAMES:
            raise ValueError(
                f"Existing output has unexpected columns: {reader.fieldnames}. "
                "Use a new output filename."
            )
        for row in reader:
            accession = row.get("uniprot", "").strip()
            if accession:
                records[accession] = row
    return records


def compact_existing(path: Path, records, input_ids: list[str]) -> None:
    if not records:
        return
    tmp_path = path.with_suffix(path.suffix + ".tmp")
    with tmp_path.open("w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=FIELDNAMES, delimiter="\t")
        writer.writeheader()
        written = set()
        for accession in input_ids:
            if accession in records:
                writer.writerow(records[accession])
                written.add(accession)
        for accession, record in records.items():
            if accession not in written:
                writer.writerow(record)
    os.replace(tmp_path, path)


def main() -> int:
    parser = argparse.ArgumentParser(
        description="Query disorder annotations and calculate fraction_IDP_IDR."
    )
    parser.add_argument("-f", "--file", required=True, help="One UniProt ID per line.")
    parser.add_argument("-o", "--output", required=True, help="Output TSV path.")
    parser.add_argument(
        "--rate-limit",
        type=float,
        default=0.5,
        help="Seconds to pause after each protein (default: 0.5).",
    )
    parser.add_argument(
        "--progress-every",
        type=int,
        default=100,
        help="Print progress every N processed proteins (default: 100).",
    )
    args = parser.parse_args()

    input_path = Path(args.file)
    output_path = Path(args.output)
    uniprots = read_ids(input_path)
    if not uniprots:
        parser.error("No UniProt IDs found in the input file.")

    output_path.parent.mkdir(parents=True, exist_ok=True)
```

```
    existing = read_existing(output_path)
    compact_existing(output_path, existing, uniprots)
    completed = {
        accession
        for accession, record in existing.items()
        if record.get("query_status") == "ok"
    }
    pending = [accession for accession in uniprots if accession not in completed]

    output_exists = output_path.exists() and output_path.stat().st_size > 0
    session = build_session()
    processed = 0
    print(
        f"Input IDs: {len(uniprots):,}; already successful: {len(completed):,}; "
        f"to query: {len(pending):,}",
        flush=True,
    )

    with output_path.open("a", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=FIELDNAMES, delimiter="\t")
        if not output_exists:
            writer.writeheader()
            handle.flush()

        try:
            for accession in pending:
                record = query_one(session, accession)
                writer.writerow(record)
                handle.flush()
                processed += 1

                if record["query_status"] != "ok":
                    print(
                        f"[WARNING] {accession}: {record['query_status']}",
                        file=sys.stderr,
                        flush=True,
                    )
                if args.progress_every > 0 and processed % args.progress_every == 0:
                    print(
                        f"Processed this run: {processed:,}/{len(pending):,}",
                        flush=True,
                    )
                if args.rate_limit > 0:
                    time.sleep(args.rate_limit)
        except KeyboardInterrupt:
            print("\nStopped safely. Rerun the same command to resume.", file=sys.stderr)
            return 130

    final_records = read_existing(output_path)
    compact_existing(output_path, final_records, uniprots)
    final_records = read_existing(output_path)
    counts = Counter(record.get("query_status", "") for record in final_records.values())
    print(f"Rows in output: {len(final_records):,}")
    print("Status counts:")
    for status, count in sorted(counts.items()):
        print(f"  {status or '<blank>'}: {count:,}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Appendix 8. get_disorder_mobidb_query.py

Location: historical_audits/get_disorder_mobidb_query.py
Provenance: Quyen's recovered sample; unchanged

Preserves the original sample disorder-query code for inspection. Its broad track union and treatment of empty responses must be checked against the production version. Use the production script for a deliberate new query; the original sample is not a mandatory replay step.

```
import time
import json
import argparse

import requests
from bioservices import UniProt


# MobiDB keywords in priority order
DISORDER_KEYWORDS = [
    'curated-disorder-priority',
    'homology-disorder-priority',
    'derived-missing_residues-priority',
    'prediction-disorder-priority',
    'derived-mobile-th_90',
]

MOBIDB_API_RATE_LIMIT_SEC = 0.5  # seconds between API calls


def get_mobidb_disordered_data(uniprot: str) -> list:
    """
    Retrieve disordered regions for a protein from MobiDB.

    Queries the MobiDB API for disorder annotations and returns a list of
    interval tuples ordered by the priority defined in DISORDER_KEYWORDS.

    Args:
        uniprot: UniProt accession ID.

    Returns:
        List of interval tuples, e.g. [([1, 56],), ([57, 68],)].
        Returns an empty list if the ID is not found or has no disorder data.
    """
    url = f'https://mobidb.org/api/download?format=json&acc={uniprot}'

    try:
        res = requests.get(url, timeout=30)
        res.raise_for_status()
    except requests.RequestException as e:
        print(f"[WARNING] MobiDB request failed for {uniprot}: {e}")
        return []

    try:
        result = res.json()
    except json.JSONDecodeError:
        print(f"[WARNING] Could not parse MobiDB JSON response for {uniprot}")
        return []

    disordered_regions = []
    for key in DISORDER_KEYWORDS:
        if key in result:
            regions = result[key]['regions']
            disordered_regions.append(tuple(regions))

    return disordered_regions


def merge_data(data_sets: list) -> tuple:
    """
    Merge and compact intervals from multiple disorder data sets.

    Combines all intervals, sorts by start position, merges overlapping
    intervals, and coalesces adjacent intervals.
```

```
    Args:
        data_sets: List of interval tuples as returned by get_mobidb_disordered_data.

    Returns:
        Tuple of merged [start, end] intervals, or an empty tuple if no
        valid intervals are present.
    """
    if not data_sets or isinstance(data_sets, str):
        return ()

    all_intervals = []
    for intervals in data_sets:
        all_intervals.extend(intervals)

    if not all_intervals:
        return ()

    # Sort by start position
    all_intervals.sort(key=lambda x: x[0])

    # Merge overlapping intervals
    merged = [list(all_intervals[0])]
    for interval in all_intervals[1:]:
        if interval[0] <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], interval[1])
        else:
            merged.append(list(interval))

    # Coalesce adjacent intervals (end + 1 == next start)
    final = [merged[0]]
    for interval in merged[1:]:
        if final[-1][1] + 1 == interval[0]:
            final[-1][1] = interval[1]
        else:
            final.append(interval)

    return tuple(final)


def count_overlap(first_range: tuple, second_ranges: tuple) -> list:
    """
    Count overlapping residues between a query range and a set of disorder regions.

    Args:
        first_range: (start, end) tuple for the query range (e.g. full protein length).
        second_ranges: Tuple of [start, end] lists representing disorder regions.

    Returns:
        List of overlap counts per region. Returns an empty list if second_ranges is empty.
    """
    if not second_ranges:
        return []

    overlaps = []
    for region in second_ranges:
        start = max(first_range[0], region[0])
        end = min(first_range[1], region[1])
        overlaps.append(max(0, end - start + 1))

    return overlaps


def get_uniprot_length(uniprot: str, u: UniProt) -> int | None:
    """
    Retrieve the sequence length of a protein from UniProt.

    Args:
        uniprot: UniProt accession ID.
        u: Initialised bioservices UniProt instance.

    Returns:
        Sequence length as an integer, or None if the lookup fails.
    """
    res = u.search(uniprot, frmt="tsv", columns="length", limit=1, database='uniprotkb')
```

```
    lines = res.strip().split('\n') if res else []

    if len(lines) < 2:
        print(f"[WARNING] Could not find length for {uniprot} in UniProt")
        return None

    try:
        return int(lines[1])
    except ValueError:
        print(f"[WARNING] Unexpected length value for {uniprot}: {lines[1]!r}")
        return None


def get_disorder_summary(uniprot: str, u: UniProt) -> dict:
    """
    Compute the disorder summary for a single protein.

    Args:
        uniprot: UniProt accession ID.
        u: Initialised bioservices UniProt instance.

    Returns:
        Dict with keys:
            'uniprot'               : accession ID
            'length'                : protein length (int) or None on failure
            'disorder_regions'      : tuple of merged [start, end] disorder intervals
            'num_disorder_residues'  : total number of disordered residues (int) or None
    """
    length = get_uniprot_length(uniprot, u)
    if length is None:
        return {
            'uniprot': uniprot,
            'length': None,
            'disorder_regions': (),
            'num_disorder_residues': None,
        }

    disorder_regions = merge_data(get_mobidb_disordered_data(uniprot))
    overlap_counts = count_overlap((1, length), disorder_regions)
    num_disorder_residues = sum(overlap_counts)

    return {
        'uniprot': uniprot,
        'length': length,
        'disorder_regions': disorder_regions,
        'num_disorder_residues': num_disorder_residues,
    }


def format_regions(disorder_regions: tuple) -> str:
    """Render merged disorder intervals as a compact string, e.g. '1-56;70-88'."""
    if not disorder_regions:
        return '-'
    return ';'.join(f"{start}-{end}" for start, end in disorder_regions)


def read_uniprot_list(path: str) -> list:
    """Read UniProt accessions from a file, one per line, skipping blank lines."""
    with open(path) as f:
        return [line.strip() for line in f if line.strip()]


def process_uniprots(uniprots: list, u: UniProt, output_file: str | None = None) -> None:
    """
    Fetch and report the disorder summary for each UniProt accession.

    Writes a tab-separated table (header + one row per accession) to stdout,
    and to output_file as well when provided. Columns:
        uniprot  disorder_regions  num_disorder_residues  length

    Args:
        uniprots: List of UniProt accession IDs.
        u: Initialised bioservices UniProt instance.
        output_file: Optional path to also write the table to.
    """
```

```
    header = "uniprot\tdisorder_regions\tnum_disorder_residues\tlength"

    out = open(output_file, 'w') if output_file else None
    try:
        print(header)
        if out:
            out.write(header + '\n')
            out.flush()

        for i, uniprot in enumerate(uniprots):
            summary = get_disorder_summary(uniprot, u)
            line = (
                f"{summary['uniprot']}\t"
                f"{format_regions(summary['disorder_regions'])}\t"
                f"{summary['num_disorder_residues']}\t"
                f"{summary['length']}"
            )
            print(line)
            if out:
                out.write(line + '\n')
                out.flush()

            # Rate-limit only between MobiDB API calls, not after the last one.
            if i < len(uniprots) - 1:
                time.sleep(MOBIDB_API_RATE_LIMIT_SEC)
    finally:
        if out:
            out.close()


if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Fetch disorder statistics (merged disorder range, number of "
                    "disordered residues, protein length) from MobiDB/UniProt for "
                    "one or more UniProt accessions."
    )
    source = parser.add_mutually_exclusive_group(required=True)
    source.add_argument("-u", "--uniprot",
                        help="A single UniProt accession ID.")
    source.add_argument("-f", "--file",
                        help="Path to a file listing UniProt accession IDs, one per line.")
    parser.add_argument("-o", "--output", default=None,
                        help="Optional output file path (tab-separated). "
                             "Results are always printed to stdout.")
    args = parser.parse_args()

    if args.uniprot:
        uniprots = [args.uniprot.strip()]
    else:
        uniprots = read_uniprot_list(args.file)

    if not uniprots:
        parser.error("No UniProt accessions found in the provided input.")

    u = UniProt()
    process_uniprots(uniprots, u, output_file=args.output)
```